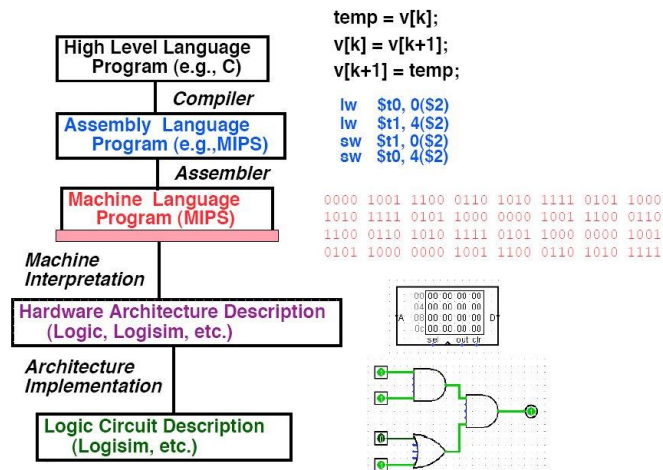


## CG2271 Real Time Operating Systems

### Tutorial 0 – Introduction to Assembly

Computers, being based on the binary number system, are inherently numerical, and can only understand numbers. For this reason, programs that we write in C, Python, JavaScript or even Java ultimately become a series of binary numbers. This binary encoding of programs is called “machine code” made up of numeric “machine instructions”. The diagram below illustrates this idea:



Because strings of numbers are very hard to read and understand, computer scientists produced an intermediate human-readable language called “assembly language”. Each instruction of assembly corresponds directly to a particular binary machine instruction.

Operating Systems necessarily operate on programs at the binary level, and since there is a 1-1 correspondence between binary instructions and assembly language, this tutorial aims to get you comfortable with assembly language as quickly as possible, so that the lecture materials are easier to understand. We will use MIPS assembly, which is much easier to understand than ARM.

Basic assembly instructions:

| Instruction                        | Meaning                                                    |
|------------------------------------|------------------------------------------------------------|
| <code>lw \$r1, var</code>          | Load contents of variable var into register \$r1           |
| <code>sw \$r1, var</code>          | Write contents of register \$r1 into variable var.         |
| <code>li \$r1, const</code>        | Load constant value const into \$r1. const can be negative |
| <code>add \$r1, \$r2, \$r3</code>  | $\$r1 = \$r2 + \$r3$                                       |
| <code>add \$r1, \$r2, const</code> | $\$r1 = \$r2 + \text{const.}$                              |
| <code>sub \$r1, \$r2, \$r3</code>  | $\$r1 = \$r2 - \$r3$                                       |
| <code>mul \$r1, \$r2, \$r3</code>  | $\$r1 = \$r2 * \$r3$                                       |
| <code>mul \$r1, \$r2, const</code> | $\$r1 = \$r2 * \text{const}$                               |
| <code>div \$r1, \$r2, \$r3</code>  | $\$r1 = \$r2 / \$r3$                                       |
| <code>div \$r1, \$r2, const</code> | $\$r1 = \$r2 / \text{const}$                               |

|                         |                                              |
|-------------------------|----------------------------------------------|
| slt \$r1, \$r2, \$r3    | \$r1 = 1 if \$r2 < \$r3, 0 if \$r2 >= \$r3   |
| slt \$r1, \$r2, const   | \$r1 = 1 if \$r2 < const, 0 if \$r2 >= const |
| beq \$r1, \$r2, target  | Branch to target if \$r1 == \$r2             |
| beq \$r1, const, target | Branch to target if \$r1 == const            |
| bne \$r1, \$r2, target  | Branch to target if \$r1 != \$r2             |
| bne \$r1, const, target | Branch to target if \$r1 != const            |
| j target                | Jump to target                               |

General Purpose Registers:

General purpose registers range from \$0 to \$31.

## Section 1. Registers and Load/Store Models

### Question 1.

Using ChatGPT or other LLMs, find out what processor general purpose registers (GPRs) are, and summarize, in your own words, what it says.

### Question 2.

Most processors and microcontrollers have a “load/store architecture”, meaning that program variables are always loaded from memory into a GPR, and operations are performed only on GPRs, with the results written back to the variable using “store instructions”. Some examples:

| C             | Assembly                                                                     | Comments                                                                                    |
|---------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| x = x + 1     | lw \$1, x<br>add \$1, \$1, 1<br>sw \$1, x                                    | # Load x into register \$1<br># Increment \$1<br># Store \$1 back into x                    |
| x = y + z - 2 | lw \$1, y<br>lw \$2, z<br>add \$2, \$1, \$2<br>sub \$2, \$2, -2<br>sw \$2, x | # Load y into \$1<br># Load z into \$2<br># \$2 = \$1 + \$2<br># \$2 = \$2 - 2<br># x = \$2 |

Rewrite the following into assembly:

x = x + 5

x = x - 1

x = x + y - z

## Section 2 Decisions

Decisions are handled using compares and branches. Some examples:

| C                                               | Assembly                                                                                                                                                                | Comments                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| if(c == 5)<br>x = x + 1;<br>else<br>y = y - 1;  | lw \$1, c<br>bne \$1, 5, else<br>lw \$2, x<br>add \$2, \$2, 1<br>sw \$2, x<br>j cont<br>else: lw \$2, y<br>add \$y, \$y, -1<br>sw \$2, y<br>cont: ...                   | # Load c into register \$1<br># Branch to else if \$1 != 5<br># IF part; load x<br># IF part; increment \$2<br># IF part, store \$2 to x<br># Exit IF part<br># ELSE part: Load y into \$2<br># ELSE part: subtract 1 from \$2<br># ELSE part: Write \$2 back to y<br># Code after if..else.. statement                                                    |
| if(c < 5):<br>x = x * 2;<br>else:<br>y = y / 2; | lw \$1, c<br>slt \$1, \$1, 5<br>beq \$1, 0, else<br>lw \$1, x<br>mul \$1, \$1, 2<br>sw \$1, x<br>j cont<br>else: lw \$1, y<br>div \$1, \$1, 2<br>sw \$1, y<br>cont: ... | # Logic: c<5 means we branch to<br># else part if c >= 5. I.e. slt \$r1, \$r2,<br>5<br># results in \$r1==0<br># IF part: Load x into \$1<br># IF part: Multiply \$1 by 2<br># IF part: Store \$1 to x<br># IF part: Jump to rest of code<br># ELSE part: Load y into \$1<br># ELSE part: Divide \$1 by 2<br># ELSE part: Store \$1 to y<br># Rest of code |

### Question 3.

Translate the following into assembly:

```
if (c >= 3)
    y = y + 1;
else:
    y = y - 1;
```

### Section 3. Loops

Loops are also implemented using comparisons and branches. In the standard assembly language design pattern for while and for statements, we test for the EXIT condition and branch to the exit point of the loop, rather than the continuation condition. We test for continuation instead for do..while statements.

The table below shows how standard C loops are typically translated to assembly:

| C                                                          | Assembly                                                                                                                                                                                    | Comments                                                                                                                                       |
|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>x = 0; while (x &lt; 5)     y = y - 2;     x++;</pre> | <pre>li \$1, 0 sw \$1, x start: slt \$2, \$1, 5       beq \$2, 0, end       lw \$2, y       add \$2, \$2, -2       sw \$2, y       add \$1, \$1, 1       sw \$1, x       j start end:</pre> | <pre># Zero x # Exit when !(x&lt;5), i.e. x&gt;=5 # Load y into \$2 # Decrement \$2 # Write back to y # Increment x # Jump back to start</pre> |
| <pre>for(x=0; x&lt;5; x++)     y = y - 2;</pre>            | Same as above                                                                                                                                                                               | Same as above                                                                                                                                  |
| <pre>x = 0; do {     y = y - 2; } while(x &lt; 5);</pre>   | <pre>li \$1, 0 sw \$1, x start: lw \$2, y       add \$2, \$2, -2       sw \$2, y       add \$1, \$1, 1       sw \$1, x       slt \$3, \$1, 5       bne \$3, 0, start</pre>                  | <pre># Initialize x to 0</pre>                                                                                                                 |

Notice how, for efficiency, the loop variable x is only written back to, and not read, since we always have the latest value in \$1.

**Question 4.**

Rewrite the following C fragment into assembly.

```
for(i=0; i<10; i++) {  
    y = y - 1;  
    if(y < 0)  
        z = z + y;  
    else  
        z = z - y;  
}
```